Final Project Report

On

Efficient Dataset Generation for Canine White Blood Cell Detection Using a Custom Convolutional Neural Network and Semi-Automated Annotation in Blood Smear Imaging

Submitted by

6510552663 Napat Praphakamol

6510552787 Varintorn Marktoum

Present to

Prof. Paruj RATANAWORABHAN

This report is part of 01204466 Deep Learning Section 450
Faculty of Engineering, Kasetsart University

1. Final Project Topic

White Blood Cell (WBC) Classification using Deep Learning — A performance comparison between a Custom CNN for white blood cell classification and YOLOv8 using microscope images.

White blood cell types classified (5 classes): Basophils, Eosinophils, Lymphocytes, Monocytes

,Neutrophils

2. Why is this Topic Interesting?

White Blood Cell Differential Count and classification are important procedures in medical diagnosis, such as for infectious diseases, leukemia, and immune deficiency disorders.

Currently, these examinations still rely heavily on laboratory specialists, which presents several limitations such as Time-consuming and requires highly skilled personnel, Susceptible to human error caused by fatigue, Shortage of medical experts in remote areas

Applying Deep Learning to this task can improve analysis speed, reduce errors, and effectively assist medical professionals. Our group chose this topic because it addresses a real-world medical problem with significant impact, and white blood cell image data is sufficiently complex to demonstrate the capabilities of Deep Learning.

3. Why Deep Learning?

Each type of white blood cell has distinct characteristics in terms of nuclear texture, cell shape, and cytoplasm color, which are highly complex and fine-grained features. Comparison with other methods:

Method	Strengths	Weaknesses
Rule-based / Thresholding	Simple and interpretable	Fails when lighting conditions or staining variations occur
Traditional ML (SVM, Random Forest)	Requires a small amount of data	Requires manual feature engineering and has lower accuracy
Deep Learning (CNN)	Automatically learns features and achieves higher accuracy	Requires large datasets and high computational resources

4. GitHub Repository Link

Github repository: https://github.com/varilltosh/DLWBC/tree/main

5. Deep Learning Architecture

This project uses a self-designed Custom CNN called **CustomWBC_CNN**, which consists of 3 Convolutional Blocks followed by Fully Connected Layers.

Layer	Operation	Output Size	Parameters
Input	RGB Image	128 × 128 × 3	—
Conv Block 1	Conv2d(3→32, k=3×3) + BN + ReLU	128 × 128 × 32	896 + 64
	MaxPool2d(2×2)	64 × 64 × 32	—
Conv Block 2	Conv2d(32→64, k=3×3) + BN + ReLU	64 × 64 × 64	18,496 + 128
	MaxPool2d(2×2)	32 × 32 × 64	—
Conv Block 3	Conv2d(64→128, k=3×3) + BN + ReLU	32 × 32 × 128	73,856 + 256
	MaxPool2d(2×2)	16 × 16 × 128	—
Flatten	16 × 16 × 128 = 32,768 nodes	32,768	—
FC1	Linear(32768→512) + ReLU	512	16,777,728
Dropout	p = 0.5	512	—
Output (FC2)	Linear(512→5)	5	2,565
Total Parameters			16,873,989

Forward Pass Equation:

$$\mathbf{x \rightarrow Block1 \rightarrow Block2 \rightarrow Block3 \rightarrow Flatten \rightarrow FC1 \rightarrow Dropout \rightarrow FC2 \rightarrow \hat{y}}$$

Function of each component:

• Conv2d: Extracts feature maps using learnable 3×3 filters

• BatchNorm: Normalizes the output of each batch to improve training stability

• ReLU: Activation function f(x)=max(0,x)f(x)=\max(0,x)f(x)=max(0,x) — enables the model to learn non-linear patterns

• MaxPool2d: Reduces the spatial dimension by half — decreases computation and helps prevent overfitting

• Dropout(0.5): Randomly disables 50% of neurons during training — helps reduce overfitting

• CrossEntropyLoss: Loss function for multi-class classification (includes Softmax internally)

6. PyTorch Code Explanation

The code is divided into 4 main sections in the train_custom_cnn.py file:

Source code: https://github.com/varilltosh/DLWBC/blob/main/Custom-CNN/train_custom_cnn.py

Part 1 — Data Pipeline

The project uses torchvision.datasets.ImageFolder to load images from folders separated by class (preview/train, preview/val, preview/test) and passes them through DataLoader to create mini-batches. The training set uses the following data augmentation techniques:

• RandomHorizontalFlip() — horizontally flips images
• RandomVerticalFlip() — vertically flips images
• RandomRotation(15) — randomly rotates images within ±15°
• ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2) — randomly adjusts color properties
• Normalize([0.5,0.5,0.5], [0.5,0.5,0.5]) — normalizes pixel values to the range [-1, 1]

Part 2 — Model Definition

The CustomWBC_CNN class inherits from nn.Module. All layers are defined in __init__(), while the forward pass is implemented in forward(). The model uses nn.Sequential to organize layers within each block.

Part 3 — Training Loop

The model is trained for 100 epochs, where each epoch consists of 2 phases:

• **Train phase:** Images are passed into the model, loss is calculated using CrossEntropyLoss (with class weights), backpropagation is performed, and weights are updated using the Adam optimizer.

• **Validation phase:** The model is evaluated using torch.no_grad() (without updating weights), validation accuracy is measured, and the best-performing model is saved.

• **LR Scheduler:** StepLR reduces the learning rate by ×0.5 every 10 epochs.

Part 4 — Evaluation

The best model from best_custom_wbc_cnn.pth is loaded and tested on the test set. The system calculates Accuracy, Precision, Recall, and F1-Score using sklearn.metrics, and generates a Confusion Matrix.

Additional File — augment_balance.py

Source code: https://github.com/varilltosh/DLWBC/blob/main/Custom-CNN/augment_balance.py

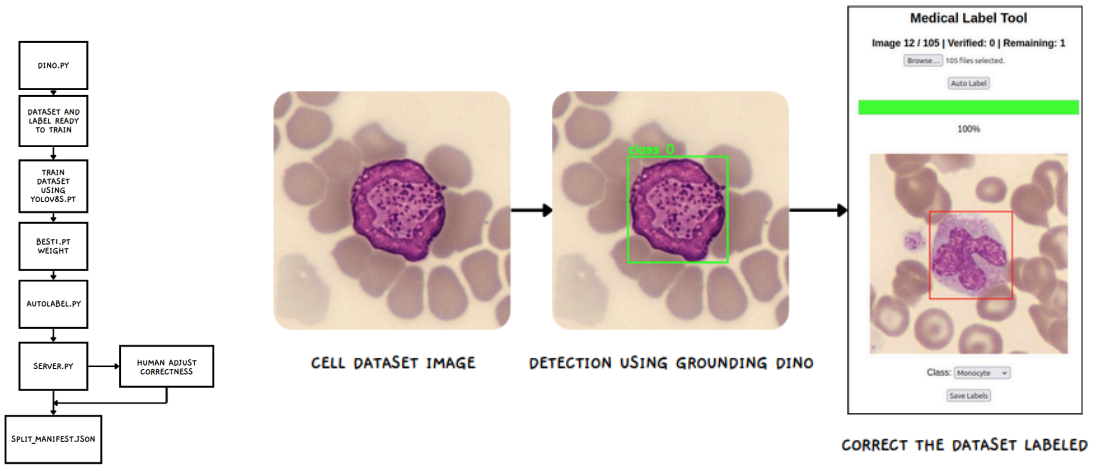
This script addresses the class imbalance problem by counting the number of images in each class and augmenting minority classes until they match the size of the largest class. The augmentation techniques include flips, 90°/180°/270° rotations, and brightness/contrast adjustments of ±15%.

7. Dataset Preparation

Dataset Source

https://github.com/varilltosh/DLWBC/tree/main/WBC_dataset_yolo
canine white blood cell dataset collected from Faculty of Veterinary Medicine Kasetsart University from laboratory

Semi-Automated labeling pipeline



The proposed system is a semi-automatic annotation pipeline for white blood cell detection and classification designed to reduce manual labeling effort while maintaining dataset quality. It begins with a raw collection of white blood cell images organized into class-specific folders, including basophils, eosinophils, lymphocytes, monocytes, and neutrophils, where no annotation is initially available in YOLO format. To address this, a foundation-model-based detection stage is applied to generate initial pseudo-labels using a vision-language or open-vocabulary detector guided by a text prompt describing the target cells. These predictions provide weak supervision by producing candidate bounding boxes that approximate the location of white blood cells without requiring human annotation.

After pseudo-label generation, the bounding boxes are filtered and converted into YOLO format by computing normalized center coordinates and dimensions relative to image size. The dataset is then systematically organized into training, validation, and testing subsets using predefined ratios with a fixed random seed to ensure reproducibility and consistent experimental evaluation. This results in a structured dataset compatible with standard object detection frameworks, following the conventional YOLO directory format for images and labels.

Once the dataset is prepared, a YOLO-based object detection model is trained using these automatically generated labels. Since the annotations originate from a weak supervision process rather than manual labeling, the model effectively learns from pseudo-labeled data, which allows scalable training without extensive human effort. The trained model weights are then saved and reused for inference in a FastAPI-based backend system.

In the next stage, the trained YOLO model is integrated into a web-based annotation tool where it performs automatic predictions on newly uploaded images. These predictions act as initial annotations that significantly speed up dataset creation. A human-in-the-loop interface allows users to review, adjust, and correct bounding boxes, ensuring that errors introduced during automatic

labeling can be refined before final storage. After verification, the annotations are exported again in YOLO format, where each label file contains normalized class ID, center coordinates, width, and height.

Finally, the system operates as an iterative feedback loop in which corrected annotations can be reintroduced into the training dataset to further improve model performance. Over time, this semi-automatic workflow reduces reliance on manual annotation while continuously enhancing dataset quality and model accuracy, making it an efficient and scalable solution for medical image labeling tasks.

Server.py
Source code: https://github.com/varilltosh/DLWBC/blob/main/semi-automated-labeling/server.py

The backend system is implemented using FastAPI to support a web-based semi-automatic annotation tool for white blood cell images. The system allows users to upload multiple images, automatically generate bounding box predictions using a pre-trained YOLO model, visualize the results in a web interface, and save corrected annotations in YOLO format for further training. When a user uploads images through the /autolabel endpoint, each image is first validated and stored in the local uploads directory. The system then applies the autolabel_image() function, which performs object detection using a YOLO model trained on previously auto-generated labels. The predicted bounding boxes and class labels for each image are extracted and combined with image metadata such as width and height. A preview version of each image is saved and exposed through a static file route so it can be displayed in the frontend. To improve user experience, the system uses a streaming response mechanism StreamingResponse to send real-time progress updates back to the frontend while processing images. After all images are processed, the final structured dataset containing image paths, predicted bounding boxes, and metadata is returned in JSON format. The /save endpoint is responsible for converting the annotated bounding boxes into YOLO label format. For each bounding box, the system calculates the center coordinates (cx, cy) and the width and height (bw, bh) relative to the image dimensions. These normalized values are then saved as text files in the format required by YOLO training: class_id cx cy bw bh .
Each image has a corresponding .txt file stored in the labels directory, ensuring compatibility with standard object detection training pipelines. Overall, this system forms a semi-automatic dataset annotation pipeline, where grounding dino predictions are used to accelerate labeling, while still allowing human verification and correction before final dataset creation. This significantly reduces manual annotation effort and improves dataset scalability for deep learning model development.

Dino.py
Source code: https://github.com/varilltosh/DLWBC/blob/main/semi-automated-labeling/dino.py

The code is implemented using a combination of GroundingDINO and Ultralytics SAM to automatically generate labeled training data for white blood cell classification and detection. The system processes a raw image dataset organized by class folders and converts it into a YOLO-compatible dataset structure with train, validation, and test splits. For each class folder (Basophils, Eosinophils, Lymphocytes, Monocytes, and Neutrophils), the images are first shuffled and split into training (70%), validation (15%), and testing (15%) subsets to ensure balanced evaluation. Each image is then processed using GroundingDINO, which performs open-vocabulary object detection based on a predefined text prompt ("purple round shape cell with nucleus"). This model generates initial bounding box predictions for potential white blood cells. The detected bounding boxes are filtered using confidence thresholds and converted from normalized coordinates into pixel space. These bounding boxes are then further refined into YOLO format by computing the center coordinates (cx, cy) and dimensions (bw, bh), normalized by image width and height. Each bounding box is assigned a class ID based on a predefined class mapping dictionary corresponding to the five white blood cell types. The processed images and their corresponding annotation files are saved into a structured dataset directory following the standard YOLO format: images/train, images/val, images/test labels/train, labels/val, labels/test This ensures compatibility with standard object detection training pipelines. A progress bar is used during execution to monitor dataset generation in real time, and a summary of generated samples per class and split is displayed at the end. Overall, this pipeline enables fully automatic dataset generation using weak supervision, where GroundingDINO provides initial pseudo-labels and the system converts them into a structured YOLO dataset. This significantly reduces manual annotation effort while maintaining a scalable and reproducible dataset creation process for deep learning model training.

Autolabel.py
Source code:
https://github.com/varilltosh/DLWBC/blob/main/semi-automated-labeling/autolabel.py

The best1.pt model is a YOLO-based object detection model trained on a dataset that was automatically generated using a foundation-model-based labeling pipeline implemented in dino.py. In this stage, the dino.py script is responsible for producing initial annotations (bounding boxes and class labels) from raw white blood cell images using a pre-trained vision-language or detection foundation model. These generated labels are then used as training data for the YOLO model. After training, the resulting weights are saved as best1.pt, which represents a model learned from the automatically constructed dataset rather than manually annotated ground truth. This makes the system a semi-automatic or weakly supervised learning pipeline, where annotation effort is significantly reduced. The trained YOLO model is then reused in the autolabel_image() function to perform inference on new images. It predicts bounding boxes and class labels for white blood cells, which can be further used for dataset expansion, verification, or correction. This creates a feedback loop where the system continuously improves dataset quality and labeling consistency through model-assisted annotation.

Spilt_manifest.json
Source code:
https://github.com/varilltosh/DLWBC/blob/main/semi-automated-labeling/split_manifest.json

The dataset was organized using a structured train–validation–test split configuration stored in a JSON metadata file. This file records the dataset creation timestamp, random seed value, split ratios, and the list of image filenames assigned to each subset for every white blood cell class. A fixed random seed (`seed = 42`) was used to ensure reproducibility, meaning the same images are consistently assigned to the same subsets across different experiments. The requested split weights were 60% for training, 10% for testing, and 20% for validation. Since the total weight equaled 90 instead of 100, the ratios were automatically normalized before splitting. The resulting dataset distribution was approximately 66.7% for training, 11.1% for testing, and 22.2% for validation. The dataset contains five white blood cell classes: Basophils, Eosinophils, Lymphocytes, Monocytes, and Neutrophils. For each class, the JSON file stores the number of samples and the corresponding filenames assigned to the train, validation, and test subsets. This structure helps prevent data leakage by ensuring that images used during training do not appear in validation or testing datasets. Such separation is especially important in medical image analysis to guarantee fair and reliable evaluation of model performance. In addition, the dataset distribution reveals a noticeable class imbalance. Classes such as Lymphocytes and Neutrophils contain significantly more samples compared to Basophils and Monocytes. For example, the Basophil class contains only a small number of images relative to the larger classes, which may negatively affect recall and F1-score during model evaluation. Minority classes are generally more difficult for deep learning models to learn effectively because the model receives fewer training examples. This imbalance can lead to biased predictions toward majority classes and unstable evaluation metrics, particularly when the test set contains only a limited number of minority-class samples. Overall, the JSON split configuration serves as a reproducible dataset manifest that ensures consistent experimentation, organized dataset management, and reliable model evaluation throughout the project.

Dataset

Dataset ratio: train,test,validate: (60%,10%,30%)

Class	Spilt	Number of Images
Basophils	train	31
	test	5
	validate	10
	Amount per class	46
Eosinophils	train	173
	test	29
	validate	58
	Amount per class	260
Lymphocytes	train	313
	test	52
	validate	104
	Amount per class	469
Monocytes	train	70
	test	12
	validate	23
	Amount per class	105
Neutrophils	train	316
	test	53
	validate	105
	Amount per class	474
5 Classes	train	903
	test	151
	validate	300
	Amount	1354

8. Training Method

Hyperparameter Tuning

The experiment compared hyperparameters across three dimensions: Learning Rate (0.001, 0.0001), Batch Size (8, 16), and Epochs (50, 100, 200), resulting in a total of 4 configurations. Each configuration was trained for 200 epochs, and model snapshots were recorded at epochs 50, 100, and 200.

Config	LR	Batch	Epoch 50 Val Acc	Epoch 100 Val Acc	Epoch 200 Val Acc	F1 @100
Config 1	0.001	8	94.35%	95.09%	95.09%	89.54%
Config 2	0.0001	8	95.33%	95.09%	94.84%	87.94%
Config 3	0.001	16	93.12%	94.10%	94.10%	86.81%
Config 4	0.0001	16	94.59%	94.35%	94.35%	85.93%

Config 1 (LR = 0.001, Batch Size = 8, Epochs = 100) was selected because it achieved the highest macro F1-score, and the validation accuracy showed no significant improvement after epoch 100, indicating that the model had already converged.

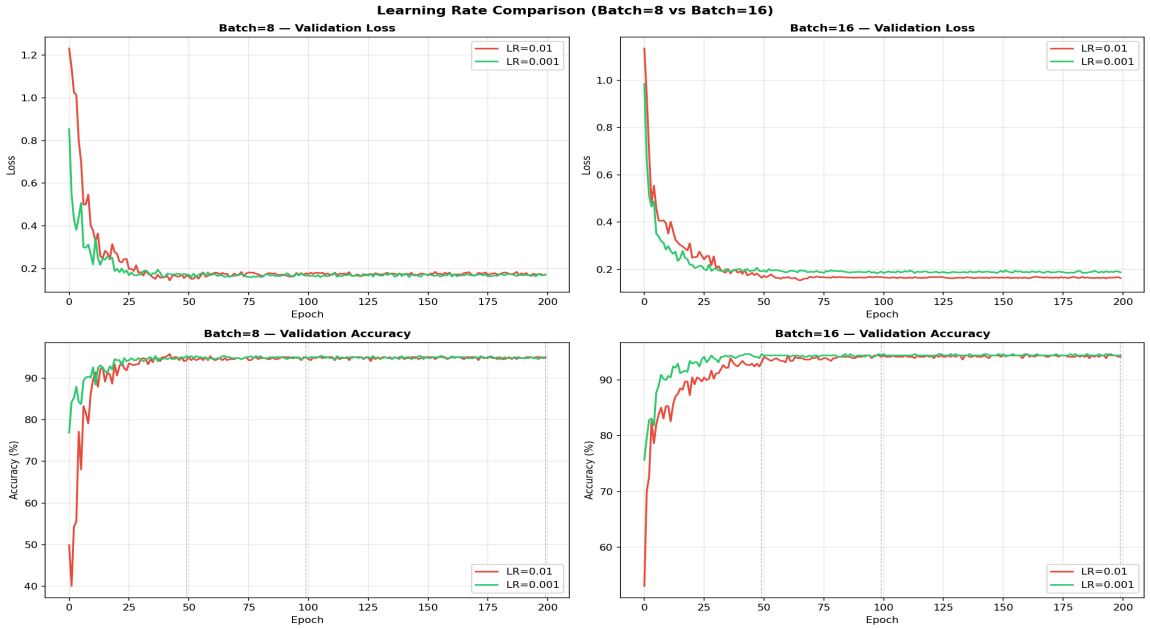


Figure 1 shows the Loss and Accuracy curves of each configuration over the full 200 training epochs.

Figure 1 presents the Training Loss, Validation Loss, Training Accuracy, and Validation Accuracy curves of all four hyperparameter configurations over 200 epochs in order to compare the effects of learning rate and batch size on model performance. Config 1 (Learning Rate = 0.001, Batch Size = 8) achieved the best overall performance. The validation accuracy increased rapidly and stabilized at approximately 95% after epoch 100, while the loss continuously decreased throughout training. This indicates that the model was able to learn white blood cell features effectively and converge quickly. Config 2, which used a lower learning rate (0.0001), showed slower learning behavior. Although its validation accuracy was close to that of Config 1, the F1-score was lower, suggesting that the model could not classify minority classes as effectively as Config 1. For Config 3 and Config 4, which used a batch size of 16, the validation accuracy was slightly lower than the configurations using batch size 8, and the loss decreased more slowly. This suggests that a larger

batch size may reduce the model's ability to generalize effectively on this dataset compared to a smaller batch size. Overall, Figure 1 demonstrates that Config 1 provided the best balance among convergence speed, validation accuracy, and macro F1-score, and was therefore selected as the final training configuration.

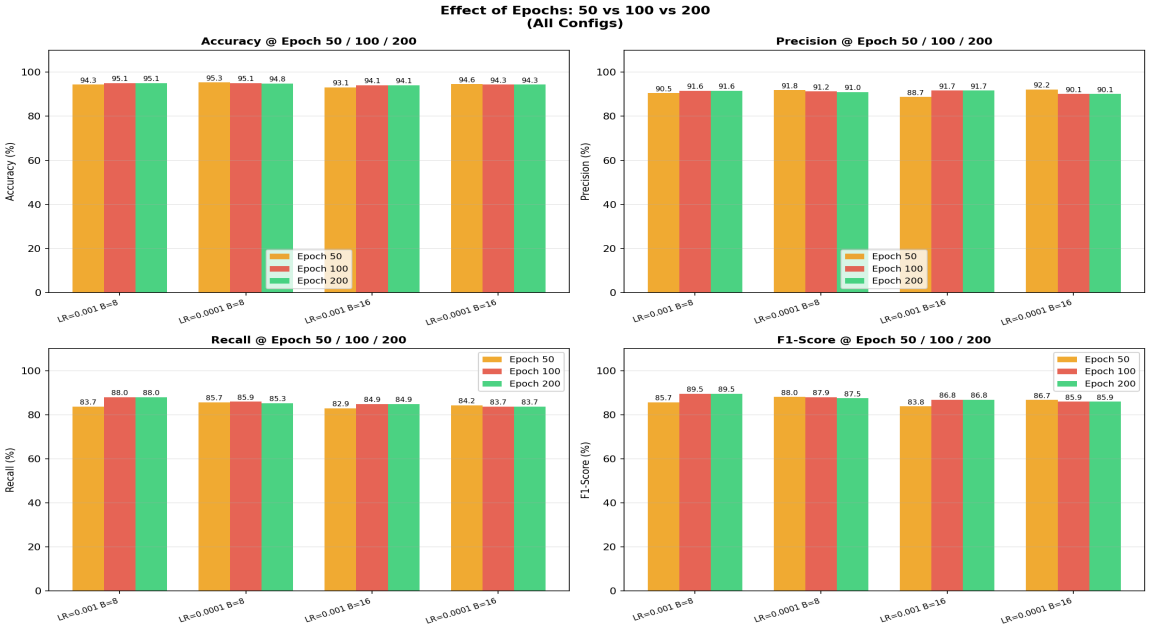


Figure 2 compares the evaluation metrics at epochs 50, 100, and 200.

Figure 2 shows that the performance of all configurations improved significantly during the early stages of training. However, after epoch 100, both accuracy and F1-score showed little additional improvement, indicating that the models had started to converge. Config 1 consistently achieved the best macro F1-score, which is particularly important for imbalanced datasets because the F1-score considers both precision and recall rather than relying solely on accuracy. Furthermore, the graphs demonstrate that continuing training up to 200 epochs did not significantly decrease accuracy, but it also did not provide meaningful performance improvement. Therefore, it can be concluded that 100 epochs were sufficient for training this model.

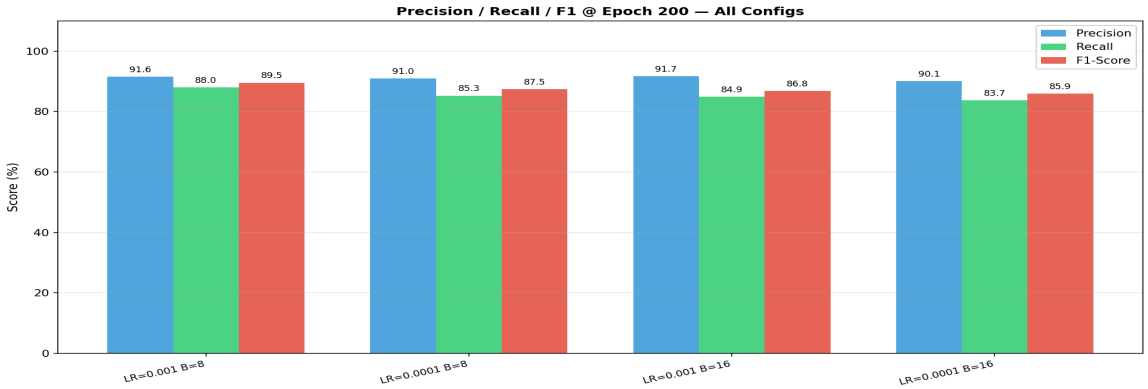


Figure 3 presents the Precision, Recall, and F1-score of all configurations at epoch 200.

The results show that Config 1 achieved the highest F1-score, indicating that the model maintained the best balance between precision and recall. Configurations using a lower learning rate or larger batch size tended to produce lower precision and recall values, particularly for some minority classes. This may be because the model could not learn sufficient features from the minority class

samples. Figure 3 further confirms that hyperparameter selection has a direct impact on the model's ability to classify white blood cells, especially in datasets with class imbalance.

Final Training Configuration

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Loss Function	CrossEntropyLoss (with class weights)
LR Scheduler	StepLR — reduces learning rate by ×0.5 every 10 epochs
Epochs	100
Batch Size	8
Image Size	128 × 128
Device	NVIDIA GeForce GTX 1050 Ti (CUDA)
Model Selection	Save the best model based on Validation Accuracy

9. Model Evaluation

Handling the Class Imbalance Problem

The dataset contains a class imbalance problem, where the **Basophils** class has significantly fewer images than the others. Three different approaches were tested to address this issue.

Medthod	Accuracy	Precision	Recall	F1 macro	Basophils Recall
① Baseline	95.62%	94.37%	85.00%	87.06%	40%
② + Class Weights	91.24%	90.91%	86.41%	87.50%	60%
③ + Balanced Augmentation	95.62%	92.38%	94.68%	93.31%	100%

Approach ③ — **Balanced Augmentation** produced the best results:

• Macro F1-score improved from **87.06% → 93.31%**

• Basophils Recall increased from **40% → 100%**

• Overall accuracy was not negatively affected

Training Curves

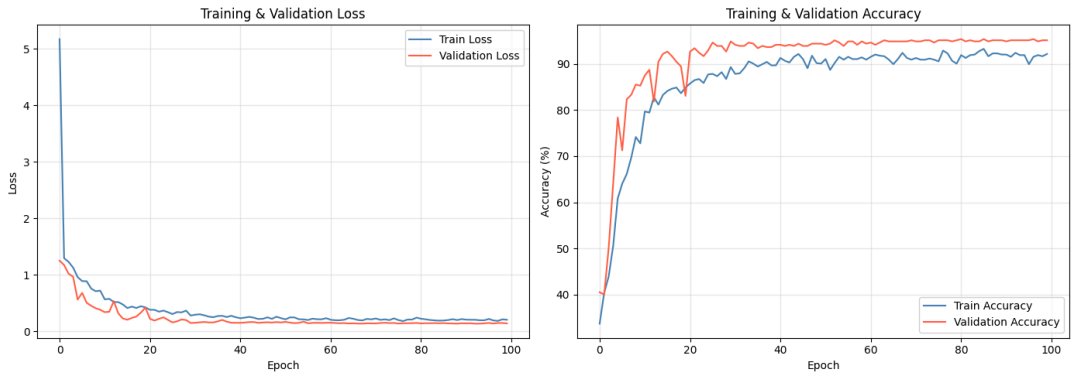


Figure 9.1: The Training & Validation Loss and Accuracy curves over 100 epochs using Baseline

Figure 9.1 shows the training and validation loss and accuracy curves using the baseline approach without any class imbalance handling technique. The graph indicates that the model achieved high training accuracy relatively quickly, while the validation accuracy improved at a slower rate and eventually plateaued. Although the overall accuracy remained high, the model struggled to correctly classify minority classes such as Basophils due to the imbalanced dataset distribution. The difference between training and validation performance suggests mild overfitting, where the model became more specialized to the training data but generalized less effectively to unseen validation samples. These results demonstrate the limitations of training directly on imbalanced medical datasets without additional balancing strategies.

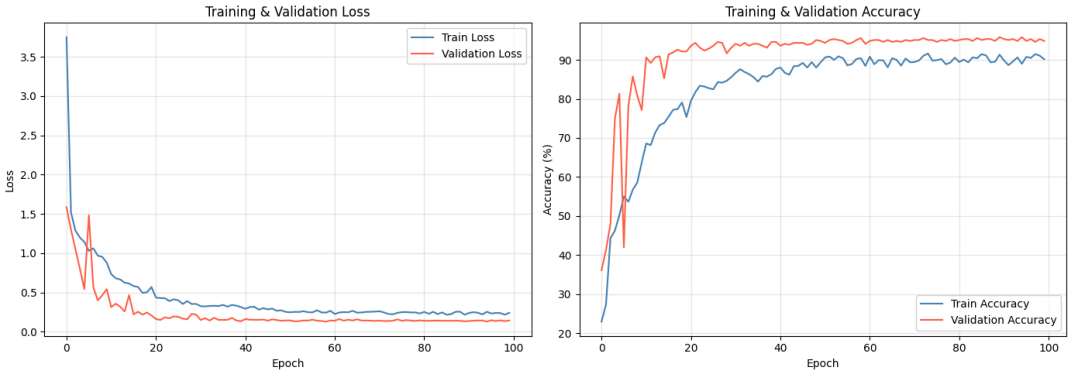


Figure 9.2: The Training & Validation Loss and Accuracy curves over 100 epochs using Class Weights

Figure 9.2 shows the training results after applying class weights in the CrossEntropyLoss function to address dataset imbalance. Compared to the baseline approach, the graph shows a more balanced validation performance and improved recall for minority classes. By assigning larger loss penalties to underrepresented classes, the model became more sensitive to minority samples during training. However, while minority class performance improved, the overall accuracy decreased slightly because the model focused more heavily on difficult classes. The curves also appear more stable than the baseline approach, indicating that class weighting helped reduce prediction bias toward majority classes.

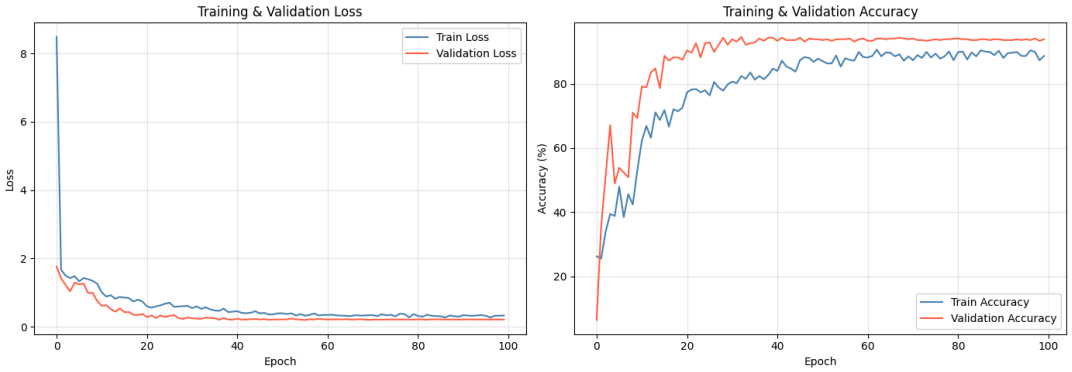


Figure 9.3: The Training & Validation Loss and Accuracy curves over 100 epochs using Balanced Augmentation

Figure 9.3 illustrates the training and validation curves using the balanced augmentation approach. The graph demonstrates smoother and more stable convergence compared to both the baseline and class-weighted methods. Validation accuracy improved significantly while validation loss decreased consistently throughout training, indicating improved generalization capability. The balanced augmentation strategy successfully increased the diversity and quantity of minority class samples, allowing the model to learn more representative feature patterns. As a result, the model achieved the highest macro F1-score and substantially improved minority class recall without negatively affecting overall accuracy. These results confirm that balanced augmentation was the most effective solution for handling the class imbalance problem in this project.

Confusion Matrix

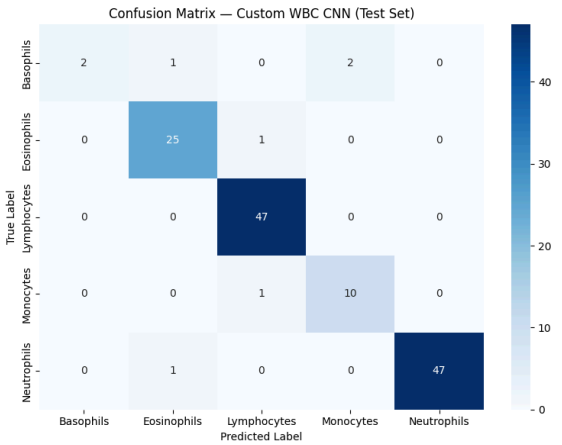


Figure 10.1 : Confusion Matrix of the Custom CNN on the Test Set using Baseline

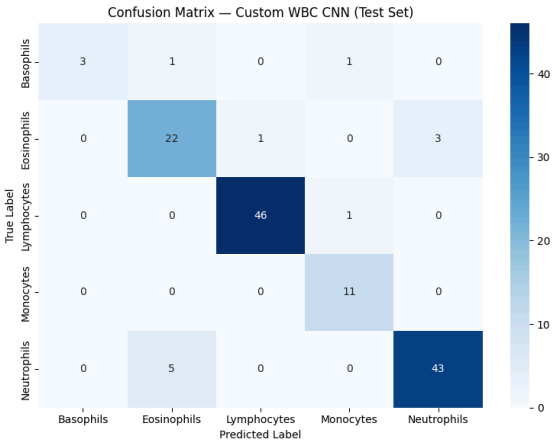


Figure 10.2 : Confusion Matrix of the Custom CNN on the Test Set using Class Weights.

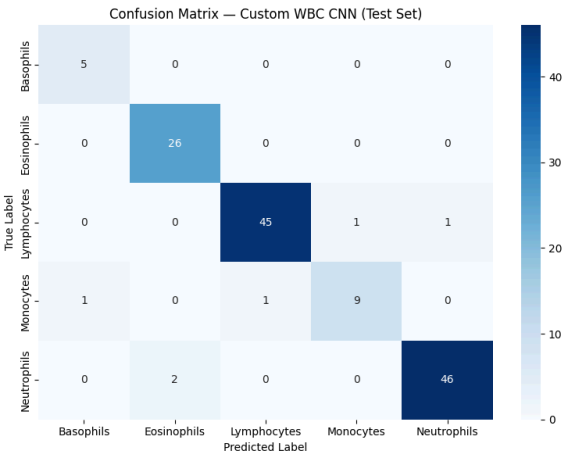


Figure 10.3 : Confusion Matrix of the Custom CNN on the Test Set using Balanced Augmentation

Figure 10.1, Figure 10.2, and Figure 10.3 compare the confusion matrices of the Custom CNN under three different training strategies: baseline training, class weighting, and balanced augmentation. The comparison clearly demonstrates how different imbalance handling techniques affected the classification performance of the model, particularly for minority classes such as Basophils and Monocytes. In Figure 10.1, which represents the baseline model without any imbalance handling method, the confusion matrix shows that the model performed well on majority classes such as Neutrophils and Lymphocytes, which contained a large number of training samples. However, minority classes experienced a higher number of misclassifications. In particular, several Basophil samples were incorrectly classified as other white blood cell types, resulting in poor recall for this class. This behavior indicates that the model became biased toward majority classes because the dataset distribution was highly imbalanced. As a result, the baseline approach achieved high overall accuracy but weaker performance for underrepresented categories. Figure 10.2 presents the confusion matrix after applying class weights during training. Compared to the baseline model, the number of correctly classified minority class samples increased noticeably, especially for Basophils. The weighted loss function forced the model to assign greater importance to minority classes during optimization, reducing prediction bias toward majority categories. Consequently, the recall of minority classes improved compared to Figure 10.1. However, some misclassifications between visually similar classes still remained, and the overall prediction distribution was not fully balanced. Although class weighting improved minority class detection, the model still struggled to generalize consistently across all classes. Figure 10.3 shows the confusion matrix produced using the balanced augmentation approach, which achieved the best overall performance among all methods. Compared to both the baseline and class-weighted models, the matrix exhibits a much stronger concentration along the diagonal, indicating significantly more correct predictions across all five white blood cell classes. Basophils achieved perfect recall, meaning all Basophil samples in the test set were classified correctly. In addition, the number of off-diagonal errors decreased substantially, demonstrating that balanced augmentation improved the model's ability to distinguish between visually similar cell types. The augmentation process increased both the quantity and diversity of minority class samples, enabling the model to learn more representative feature patterns and improve generalization capability.

Overall, the comparison between Figure 10.1, Figure 10.2, and Figure 10.3 clearly shows that balanced augmentation was the most effective method for addressing the class imbalance problem. While class weighting partially improved minority class performance, balanced augmentation produced the most balanced confusion matrix, the fewest misclassifications, and the highest overall classification consistency across all white blood cell categories.

Model Comparison (Before Balanced Data) : Custom CNN vs YOLOv8

	Batch size	Epochs	Learning rate	Accuracy	Precision	Recall	F1-score
Custom CNN (unbalanced)	8	50	0.001	94.35%	90.54%	83.71%	85.68%
			0.0001	95.33%	91.83%	85.67%	88.03%
		100	0.001	95.09%	91.58%	87.99%	89.54%
			0.0001	95.09%	91.25%	85.92%	87.94%
		200	0.001	95.09%	91.58%	87.99%	89.54%
			0.0001	94.84%	90.98%	85.28%	87.45%
	16	50	0.001	93.12%	88.75%	82.89%	83.77%
			0.0001	94.59%	92.19%	84.24%	86.72%
		100	0.001	94.10%	91.66%	84.85%	86.81%
			0.0001	94.35%	90.08%	83.70%	85.93%
		200	0.001	94.10%	91.66%	84.85%	86.81%
			0.0001	94.35%	90.08%	83.70%	85.93%

Figure 11.1 : Custom CNN unbalanced data performance

	Batch size	Epochs	Learning rate	Accuracy	Precision	Recall	F1-score
Yolov8s.pt (unbalanced)	8	50	0.001	90.56%	92.63%	88.48%	90.51%
			0.0001	92.11%	94.24%	89.97%	92.06%
		100	0.001	93.18%	93.96%	92.40%	93.17%
			0.0001	91.19%	89.94%	92.43%	91.17%
		200	0.001	91.77%	94.49%	89.05%	91.69%
			0.0001	91.85%	89.69%	94.00%	91.79%
	16	50	0.001	90.56%	90.16%	90.95%	90.55%
			0.0001	92.27%	93.09%	91.44%	92.26%
		100	0.001	91.61%	91.38%	91.84%	91.61%
			0.0001	91.34%	92.46%	90.22%	91.33%
		200	0.001	92.09%	91.68%	92.49%	92.08%
			0.0001	92.80%	93.16%	92.44%	92.80%

Figure 11.2 : Yolov8s.pt unbalanced data performance

The comparison between the Custom CNN and YOLOv8 before applying balanced augmentation highlights the impact of dataset imbalance on model performance. Before balancing the dataset, both models experienced performance limitations, particularly in minority classes such as Basophils and Monocytes. The Custom CNN still achieved stronger overall classification performance than YOLOv8, demonstrating better feature learning capability for white blood cell morphology. However, the imbalance in the training dataset caused the models to become biased toward majority classes such as Neutrophils and Lymphocytes, which contained significantly more training samples. As a result, minority classes showed lower recall and F1-score values due to insufficient representation during training. YOLOv8 was more strongly affected by this imbalance because the model simultaneously performs object detection and classification, making the learning process more difficult when minority samples are limited. The comparison before balanced augmentation clearly demonstrates that dataset imbalance negatively affects model generalization and classification consistency. These observations also emphasize the importance of balanced augmentation, which later improved minority class performance and significantly increased the overall F1-score of the Custom CNN.

Final Test Results — Custom CNN (Balanced Augmentation)

Class	Precision	Recall	F1-Score	Support
Basophils	83.33%	100.00%	90.91%	5
Eosinophils	92.86%	100.00%	96.30%	26
Lymphocytes	97.83%	95.74%	96.77%	47
Monocytes	90.00%	81.82%	85.71%	11
Neutrophils	97.87%	95.83%	96.84%	48
Macro Avg	92.38%	94.68%	93.31%	137

The final test results of the Custom CNN trained using the balanced augmentation approach demonstrate strong and consistent classification performance across all five white blood cell classes. The model achieved an overall macro precision of 92.38%, recall of 94.68%, and F1-score of 93.31%, indicating that the network was able to accurately classify white blood cell images while maintaining balanced performance between prediction correctness and detection capability. Among all classes, Eosinophils, Lymphocytes, and Neutrophils achieved particularly high F1-scores above 96%, showing that the model successfully learned highly discriminative features for these categories. Basophils achieved a recall of 100%, meaning all Basophil samples in the test set were correctly detected. Although the precision for Basophils was slightly lower due to a small number of false positive predictions, the overall F1-score remained high at 90.91%. Monocytes produced the lowest F1-score among all classes at 85.71%, which may be caused by the relatively small number of training samples and visual similarities with other white blood cell types. Overall, the results confirm that balanced augmentation significantly improved minority class performance while maintaining high overall classification accuracy and generalization ability.

Model Comparison (Balanced Augmentation) : Custom CNN vs YOLOv8s

Metric	Custom CNN	YOLOv8s
Task	Classification	Object Detection
Architecture	3-Block Custom CNN	YOLOv8 (pretrained)
Epochs	100	100
Accuracy	95.62%	85.08%
Precision (macro)	92.38%	86.47%
Recall (macro)	94.68%	83.69%
F1-Score (macro)	93.31%	85.06%

The comparison between the Custom CNN and YOLOv8s after applying balanced augmentation shows that the Custom CNN consistently outperformed YOLOv8s across all evaluation metrics. The Custom CNN achieved an overall accuracy of 95.62%, while YOLOv8s achieved 85.08%. Similarly, the Custom CNN obtained higher macro precision, recall, and F1-score values of 92.38%, 94.68%, and 93.31%, compared to YOLOv8s's 86.47%, 83.69%, and 85.06%, respectively. These results indicate that the Custom CNN produced more reliable and balanced predictions across all white blood cell classes. One important reason for this difference is that the Custom CNN was specifically designed for image classification tasks, allowing the model to focus entirely on extracting detailed morphological features from white blood cell images. In contrast, YOLOv8s performs both object localization and classification simultaneously, which introduces additional complexity and may reduce classification performance when working with relatively small datasets. Furthermore, the balanced augmentation strategy appears to benefit the Custom CNN more effectively by improving minority class representation and reducing prediction bias.

Overall, the results demonstrate that the proposed Custom CNN architecture is more suitable for fine-grained white blood cell classification than YOLOv8s under the conditions of this study.

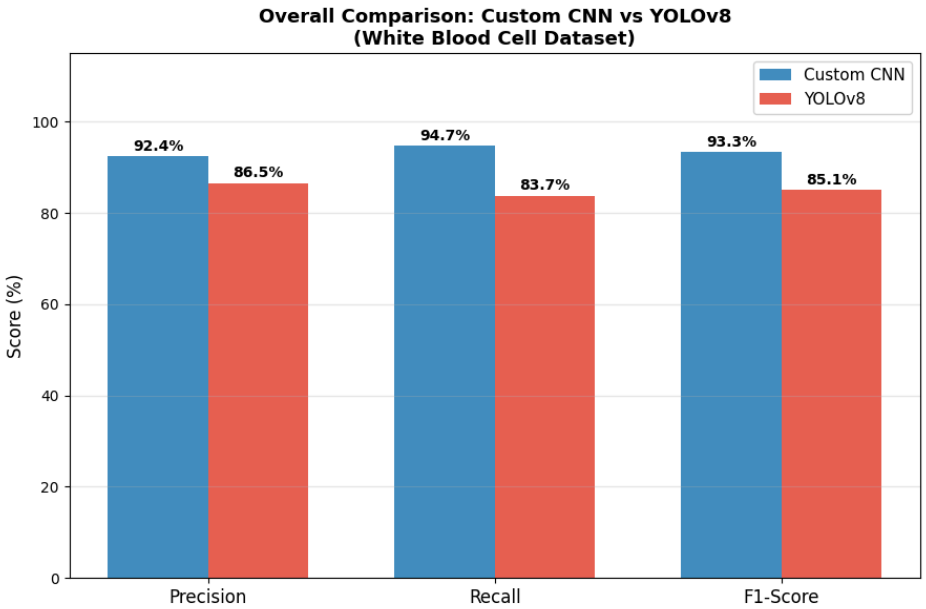


Figure 13.1: Overall Precision/Recall/F1 — Custom CNN vs YOLOv8

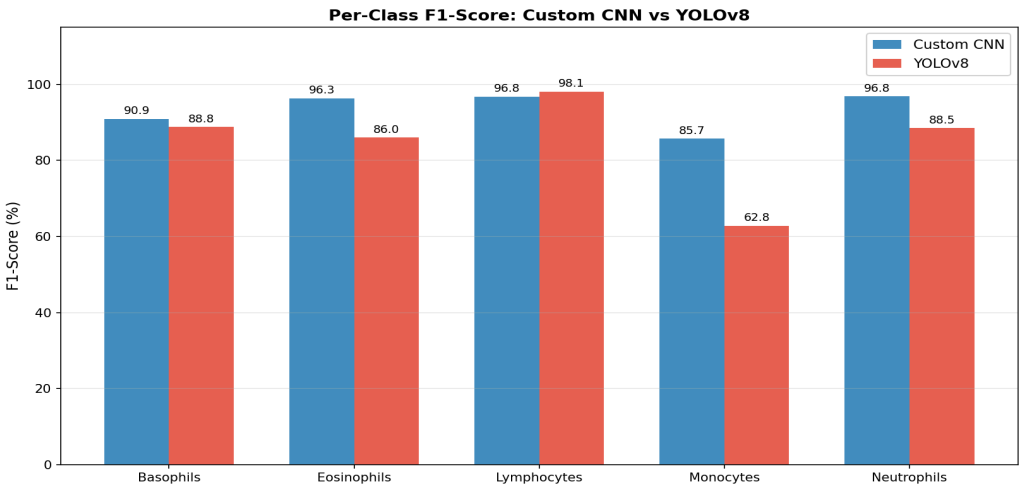


Figure 13.2: Per-Class F1-Score — Custom CNN vs YOLOv8

From Figure 13.1, the Custom CNN achieved higher overall performance than YOLOv8 in all metrics. The Custom CNN obtained a macro precision of 92.38%, recall of 94.68%, and F1-score of 93.31%, whereas YOLOv8 achieved lower scores of 86.47%, 83.69%, and 85.06%, respectively. These results indicate that the Custom CNN produced more accurate predictions with fewer false positives and fewer missed detections. The higher recall of the Custom CNN also suggests that it was more effective at recognizing white blood cell samples across all classes, particularly minority classes. In contrast, YOLOv8 showed lower recall and F1-score, indicating weaker overall classification consistency. Figure 9 further supports these observations by comparing the per-class F1-scores of both models. The graph shows that the Custom CNN consistently outperformed YOLOv8 across nearly all white blood cell categories. The performance gap was especially noticeable in minority classes such as Basophils and Monocytes, where the Custom CNN achieved substantially higher F1-scores. This improvement demonstrates that the balanced augmentation strategy effectively helped the Custom CNN learn more representative features for underrepresented classes. On the other hand, YOLOv8 exhibited larger variations in performance between classes, suggesting that the model was more sensitive to dataset imbalance. Another important comparison between the two figures is that Figure 13.1 presents the overall average behavior of the models, while Figure 13.2 reveals the distribution of performance across individual classes. Although YOLOv8 still achieved reasonable performance in majority classes such as Neutrophils and Lymphocytes, Figure 13.2 shows that its performance decreased significantly for minority classes. In contrast, the Custom CNN maintained consistently high F1-scores across both majority and minority categories, demonstrating stronger generalization capability and more balanced learning behavior. Overall, both Figure 13.1 and Figure 13.2 clearly indicate that the proposed Custom CNN combined with balanced augmentation achieved superior classification performance compared to YOLOv8. The Custom CNN provided better overall accuracy, more balanced per-class performance, and greater robustness against dataset imbalance, making it more suitable for fine-grained canine white blood cell classification tasks.

10. Reference Articles and Related Work

1. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.

2. Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767.

3. Jocher, G. et al. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

4. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25.

5. Shahin, A. I., Guo, Y., Amin, K. M., & Sharawi, A. A. (2019). White blood cells identification system based on convolutional deep neural learning networks. Computer Methods and Programs in Biomedicine, 168, 69–80.

6. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. CVPR, 770–778.

7. Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training. ICML.

8. Srivastava, N. et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. JMLR, 15(1), 1929–1958.

11. Group Contribution

Member	Role & Tasks	Contribution
6510552660 Napat Praphakamol	Custom CNN design, Hyperparameter tuning, Balanced Augmentation, Class Weights, Evaluation, Graph generation	50%
6510552787 Varintorn Marktoum	YOLOv8 training, YOLO evaluation, Hyperparameter tuning, Evaluation ,Comparison analysis, Github repository creation	50%